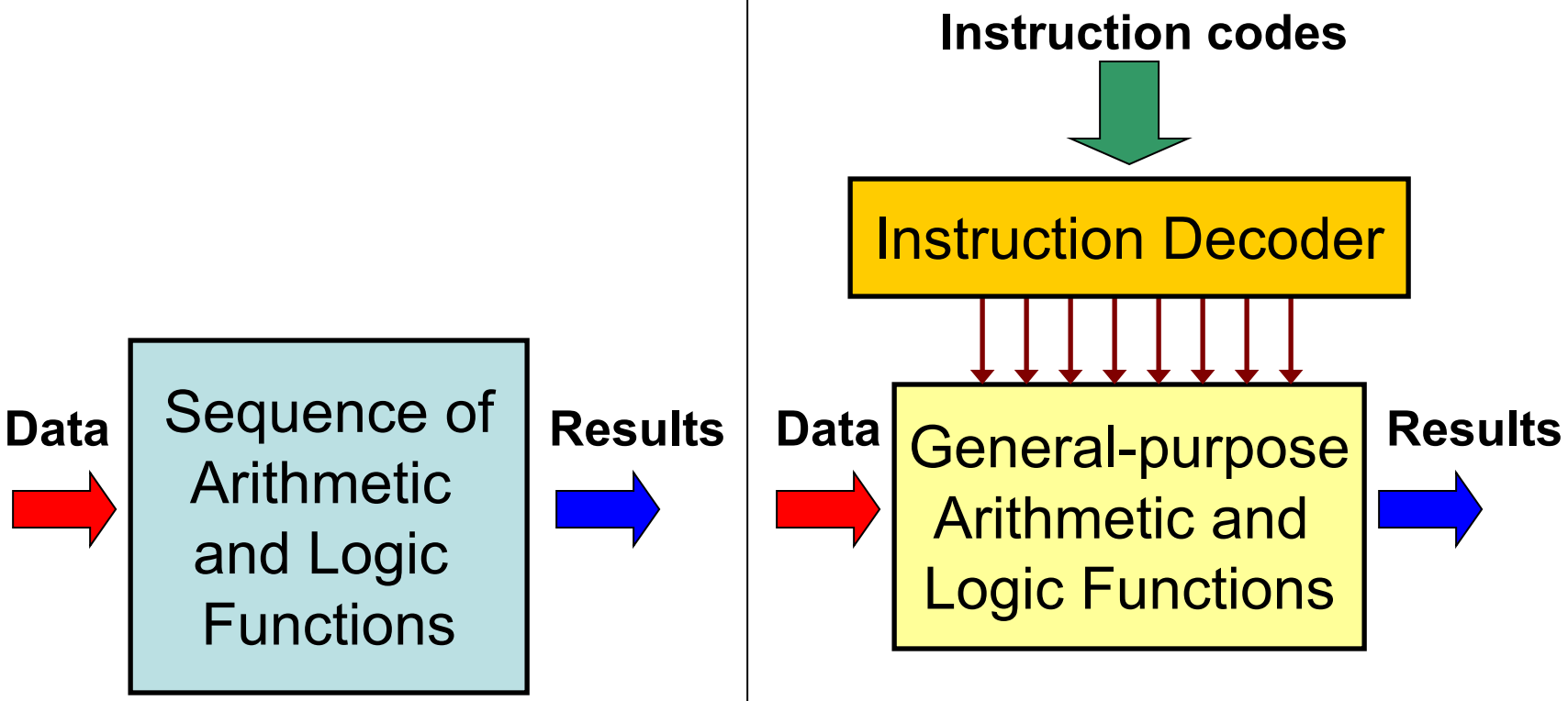
Chapter 2

Data Organisation in Memory

Role of Memory in Computing

Approaches to Computing



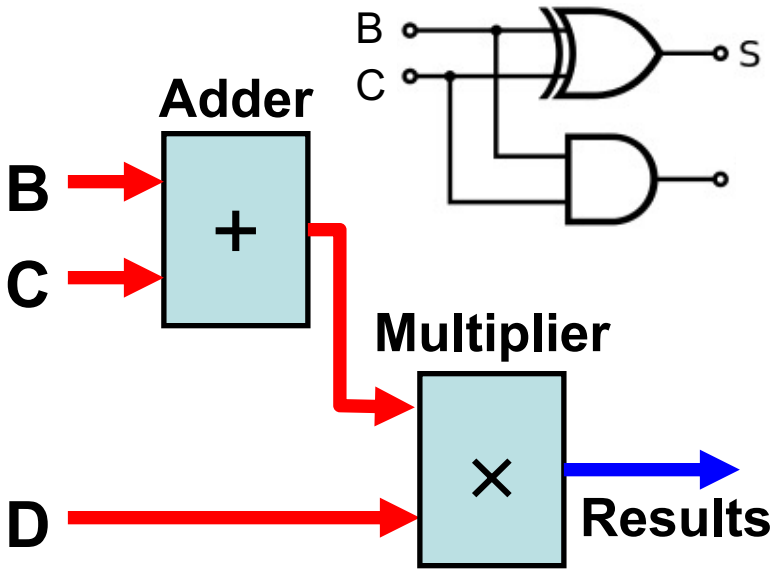
Programming in Hardware

Programming in Software

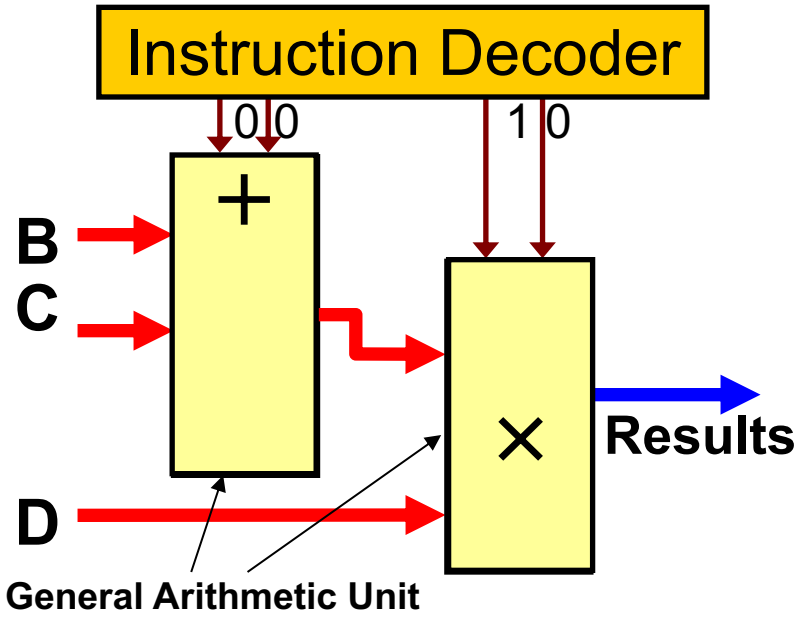
Approaches to Computing

Implementing A = (B+C)×D

Fast computation but very inflexible



Slower and more complex but easily programmable



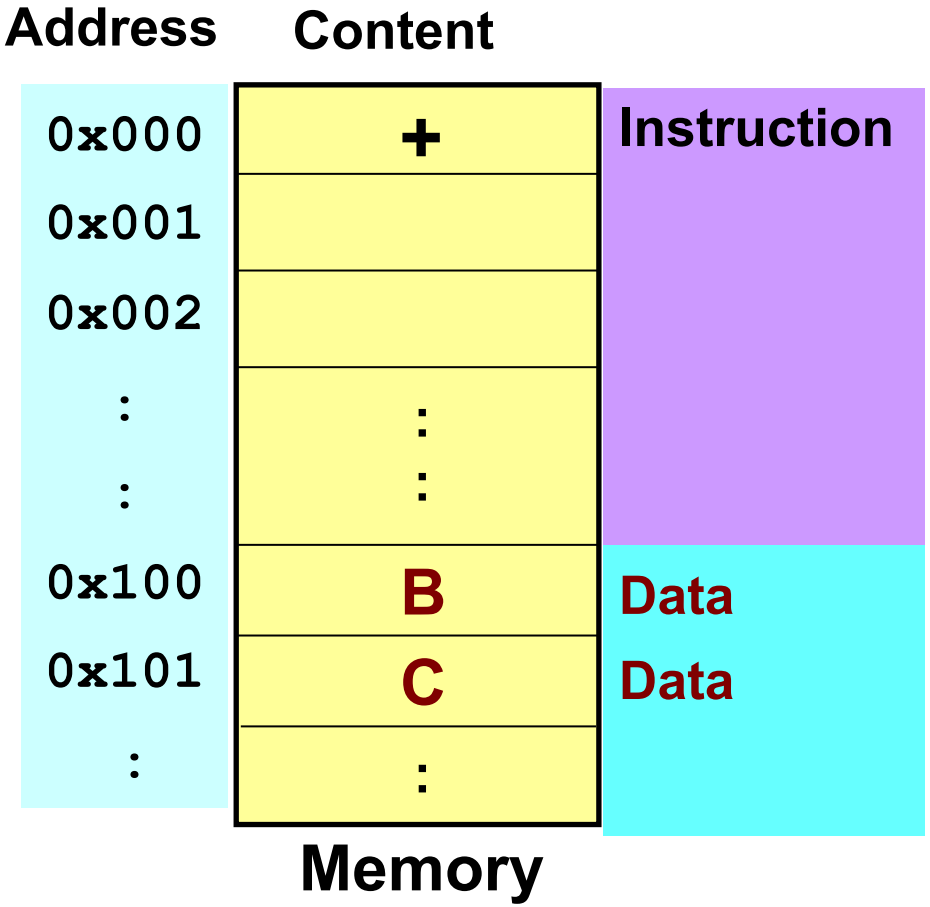
Programming in Hardware

Programming in Software

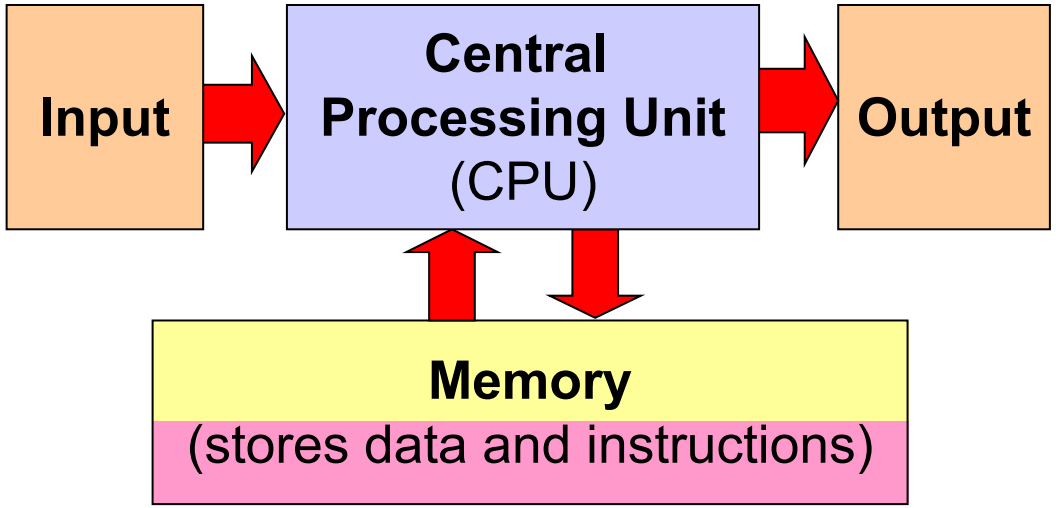
Code, Data and Memory

- What is code and what is data?
  - **Code** is a sequence of instructions.
  - **Data** are values these instructions operate on.
- What is the memory?
  - Its a sequential list of addressable storage elements for storing both instructions and data.

e.g. B+C



The Stored Program Concept



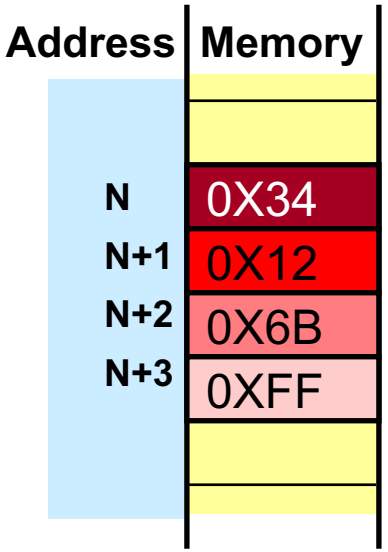
- Most modern day computer design are based on von Neumann's stored program concept:

1. Both data & instructions are stored in the same memory
2. Contents of memory are addressable by location, without regard to data type
3. Execution occurs sequentially (unless explicitly modified)

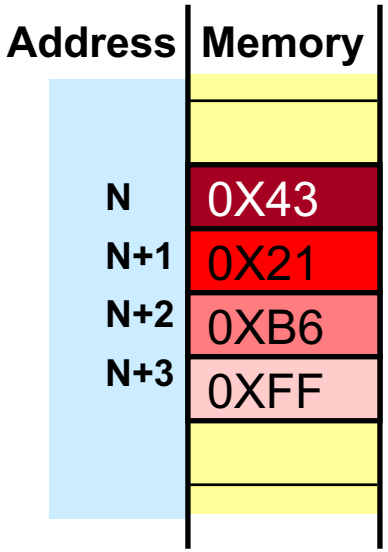
For the shown integer declaration, how will it be stored in memory assuming little-endian?

```
main()
{
 int x;    //declare an integer variable

 x = 0x34126BFF;

}
```

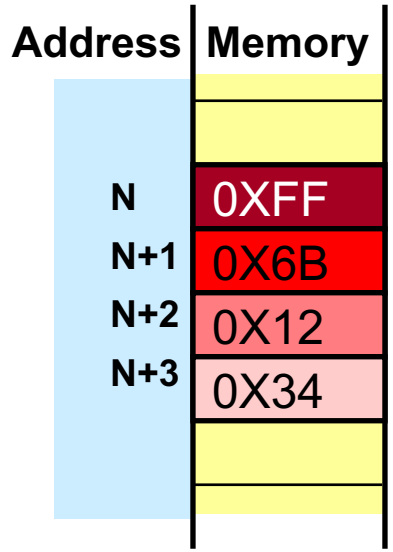
(A)



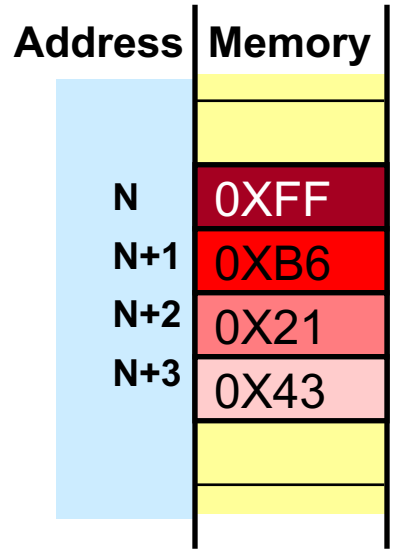
(B)



✓ (C)

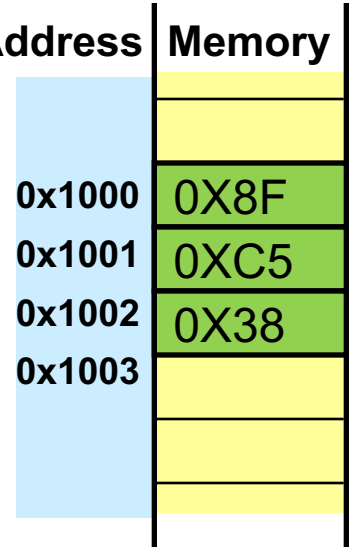


(D)



For the shown memory content, what is the value of a possible negative short integer assuming little endian?

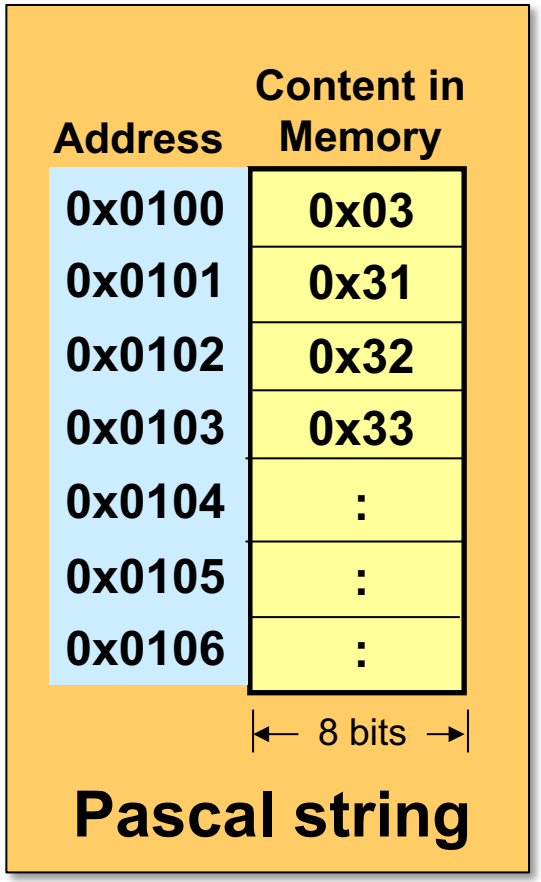
- A) 0x8FC5
- ✓ B) 0xC58F
- C) 0XC538
- D) 0X835C



For a Pascal String, what is the maximum length of the string?

```
main()
{
 char s[4]="123";  //a string constant
}
```

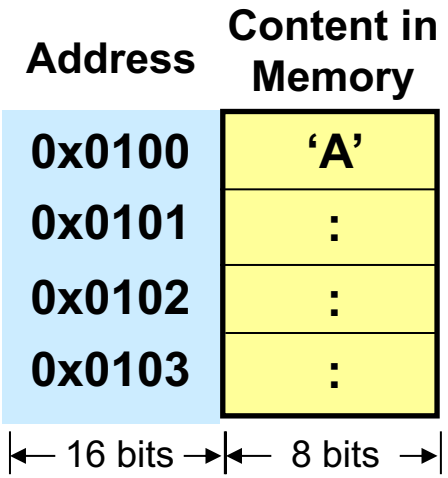
- A) 256
- ✓ B) 255
- C) 128
- D) No limit



What is the size of the pointer shown in the code segment for the shown memory?

- A) 1 Byte
- ✓ B) 2 Bytes
- C) 3 bytes
- D) 4 Bytes

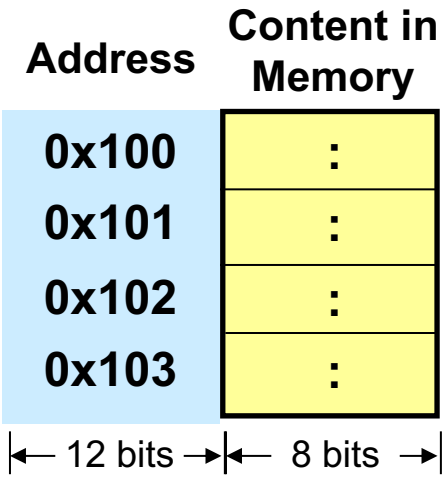
```
main()
{
 char c ;        //char variable c
 char *ptr;      //char pointer ptr
 c = 'A';        //assign value 'A' to c
 ptr = &c;       //ptr gets address of c
}
```



What is the size of the pointer shown in the code segment for the shown memory?

- A) 8 bits
- B) 12 bits
- ✓ C) 16 bits
- D) 24 bits

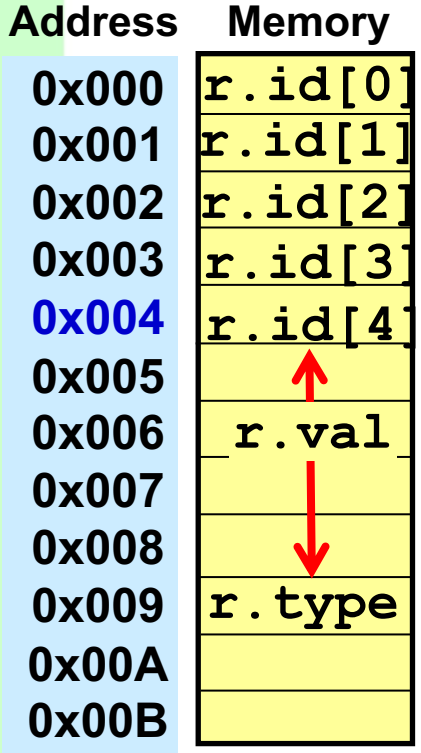
```
main()
{
 short int i ;
 Short int *ptr;
 i = 1500;
 ptr = &i;     //ptr gets address of c
}
```



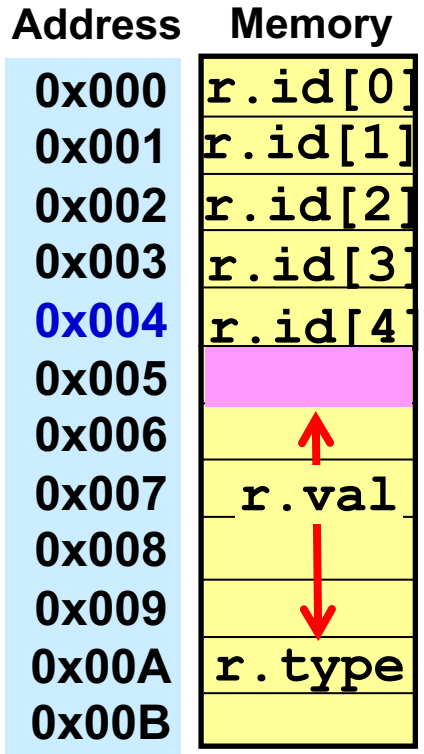
For the shown struct, what is the correct padding for a 32-bit word machine?

```
struct data {
  char id[5];
  int val;
  char type;
}
rec1 r;
```

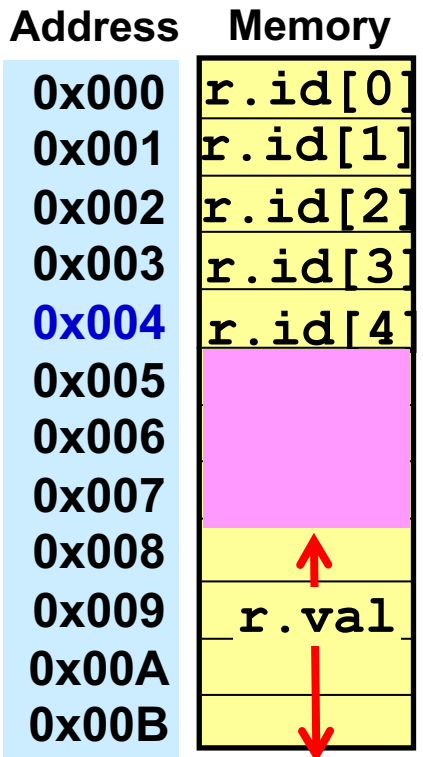
(A)



(B)



✓ (C)



(D)